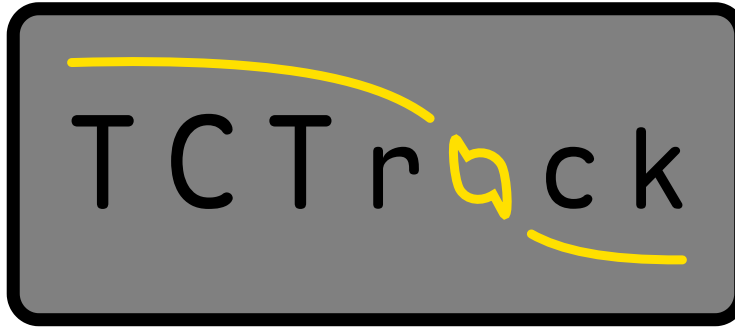




TCTrack: Working with researchers to enable FAIR tropical cyclone tracking

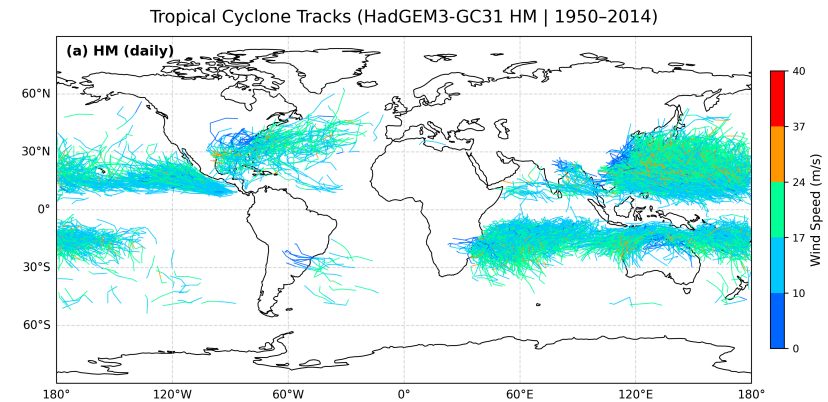
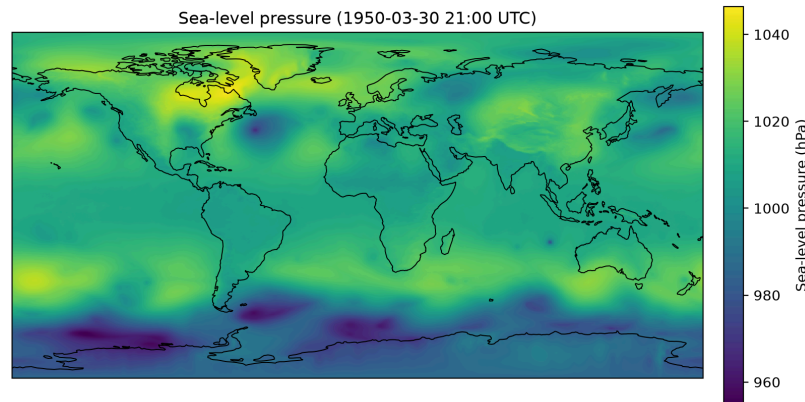
Sam Avis Jack Atkinson Simon Sadler Surbhi Goel Charles Powell
Alison Ming

Institute of Computing for Climate Science, University of Cambridge, UK

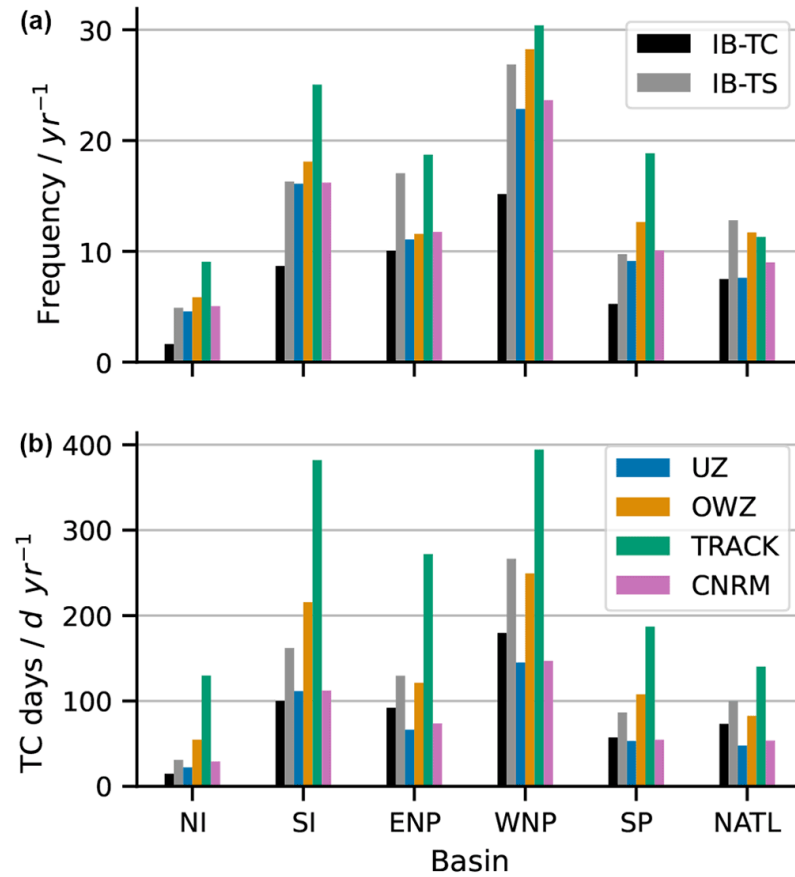
2026-09-10

Topic

Identifying tropical cyclone tracks from gridded data
(simulations, forecasts, historical reanalysis data)

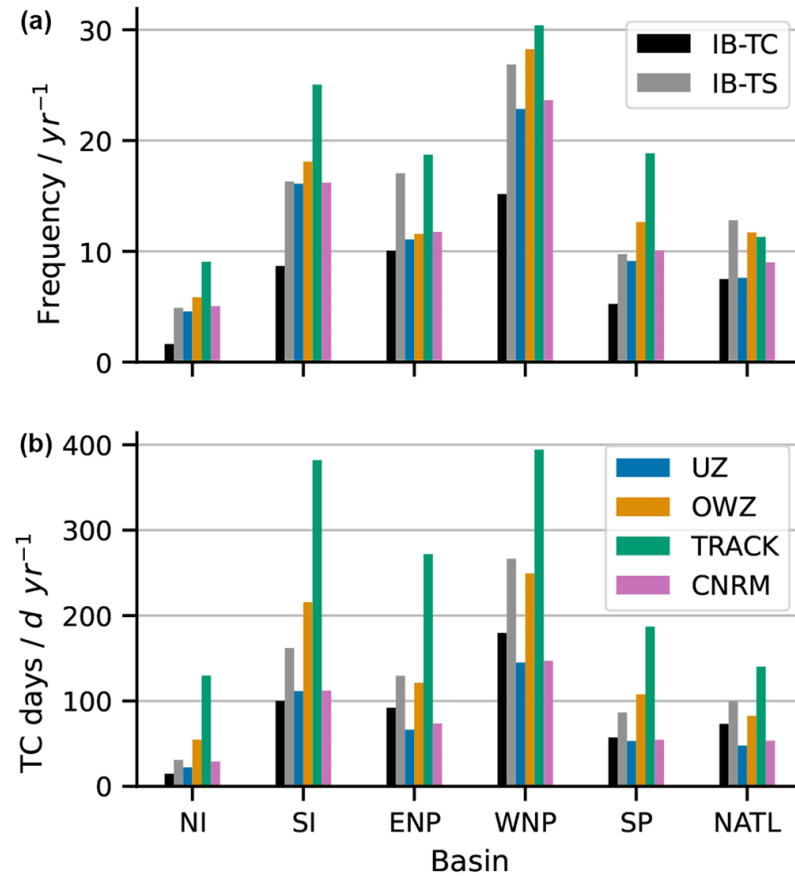


The research problem

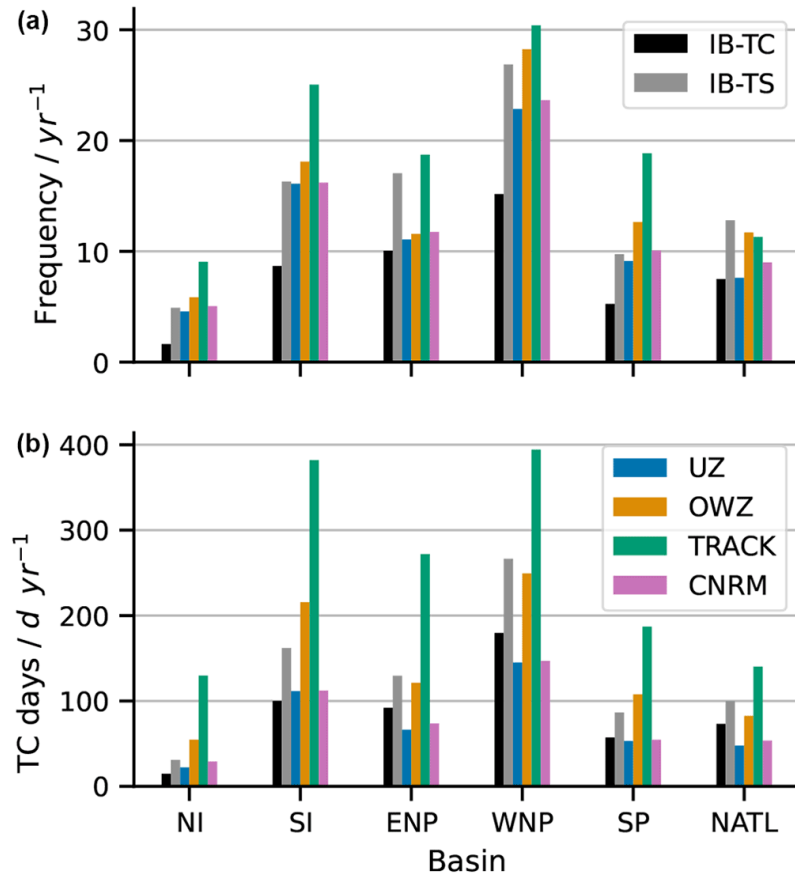


The research problem

- Multiple tracking algorithms exist, but give different results.

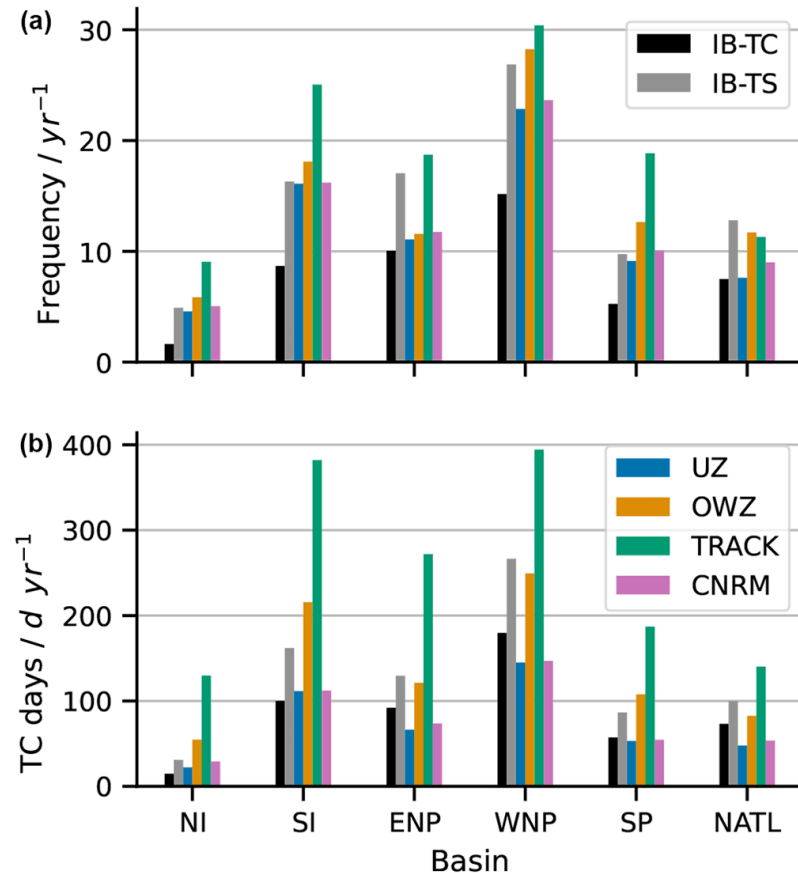


The research problem



- Multiple tracking algorithms exist, but give different results.
- Each are different and difficult to learn – many researchers will stick with just one.

The research problem



- Multiple tracking algorithms exist, but give different results.
- Each are different and difficult to learn – many researchers will stick with just one.
- We want a code that implements them with a unified interface.

Applying RSE expertise - Don't reinvent the wheel

Initial plan: Reimplement tracking algorithms in Python - researchers did not have full knowledge of existing domain software.

Also, existing codes were not necessarily well documented / work on all systems.

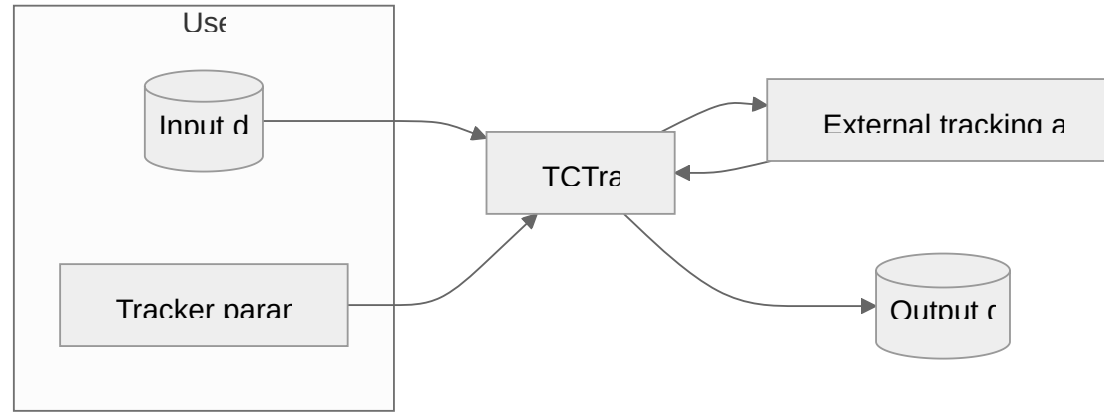
Applying RSE expertise - Don't reinvent the wheel

Initial plan: Reimplement tracking algorithms in Python - researchers did not have full knowledge of existing domain software.

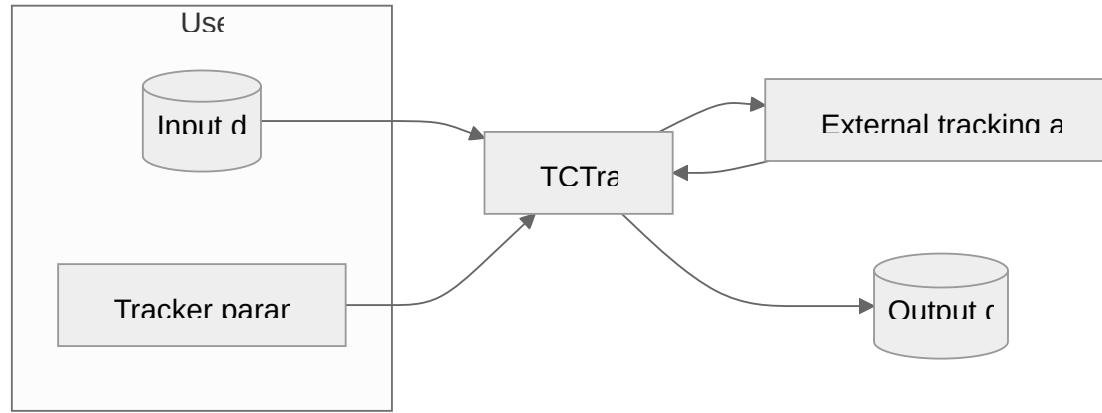
Also, existing codes were not necessarily well documented / work on all systems.

New plan: Understand and build on existing codes to improve code sustainability — RSE expertise needed.

Applying RSE expertise - The framework

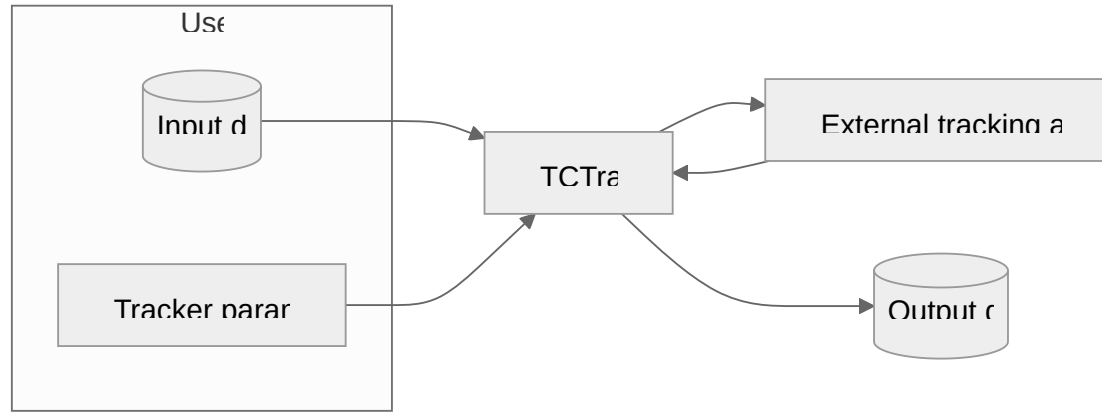


Applying RSE expertise - The framework



- TCTrack only needs an interface and parameters for each tracking algorithm.

Applying RSE expertise - The framework



- TCTrack only needs an interface and parameters for each tracking algorithm.
- Easily extensible to encourage community contributions.

Building domain experience

To wrap the tracking algorithms, we had to gain an understanding of them rather than treating them as black boxes.

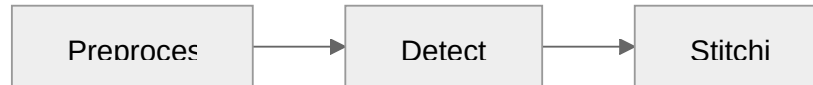
We were able to identify themes across algorithms.

Building domain experience

To wrap the tracking algorithms, we had to gain an understanding of them rather than treating them as black boxes.

We were able to identify themes across algorithms.

e.g. Broadly 3 main steps:



Building domain experience

We want to expose as many parameters and functionality from the underlying code as possible.

```
detect_params = te.TEDetectParameters(  
    search_by_min="msl",  
    time_filter="6hr",  
    merge_dist=6.0,  
    closed_contours=[  
        te.TEContour(var="msl", delta=200.0, dist=5.5),  
        te.TEContour(var="_DIFF(zg(300hPa),zg(500hPa))", delta=-6.0, dist=6.5, minmaxdist=1.0),  
    ],  
    output_commands=[  
        te.TEOutputCommand(var="msl", operator="min"),  
        te.TEOutputCommand(var="orog", operator="max"),  
        te.TEOutputCommand(var="si10", operator="max", dist=2.0),  
    ],  
)  
  
stitch_params = te.TEStitchParameters(  
    caltype="360_day",  
    max_sep=8.0,  
    min_time="54h",  
    max_gap="24h",  
    min_endpoint_dist=8.0,  
    threshold_filters=[  
        te.TETHreshold(var="latitude", op="<=", value=50, count=10),  
        te.TETHreshold(var="latitude", op=">=", value=-50, count=10),  
        te.TETHreshold(var="orog", op="<=", value=150, count=10),  
        te.TETHreshold(var="si10", op=">=", value=10, count=10),  
    ],  
)
```

Building domain experience

At the same time, we don't want it to be overwhelming and keep it simple to use.

E.g. Using default parameter sets:

```
detect_params, stitch_params = te.parameter_set("UZ")
```

To do this we need a good understanding of how the codes work and how the algorithms are used in research.

FAIR outputs: the problem

```
start      2 1950      1      4      0
    54.67 -22.15    16.65  999.23    0.00027446
    55.72 -24.49    17.38  997.83    0.00031078
start     13 1950      1      6      0
    67.32  -9.26    17.90 1002.11    0.00065708
    67.68  -9.26    15.32 1003.88    0.00044106
...
```

```
track_id, year, month, day, hour, i, j, lon, la
0, 1950, 1, 1, 6, 572, 320, 201.269531, -14.882
0, 1950, 1, 1, 12, 569, 318, 200.214844, -15.35
0, 1950, 1, 1, 18, 567, 317, 199.511719, -15.58
0, 1950, 1, 2, 0, 565, 320, 198.808594, -14.882
...
```

```
netcdf ff_trs.test_out {
dimensions:
    tracks = 703 ;
    record = UNLIMITED ; // (78 currently)
variables:
    int TRACK_ID(tracks) ;
        TRACK_ID:add_fld_num = 0 ;
        TRACK_ID:tot_add_fld_num = 0 ;
        TRACK_ID:loc_flags = "" ;
    int FIRST_PT(tracks) ;
```

FAIR outputs: the problem

```
start      2 1950      1      4      0
  54.67 -22.15  16.65 999.23  0.00027446
  55.72 -24.49  17.38 997.83  0.00031078
start     13 1950      1      6      0
  67.32  -9.26  17.90 1002.11  0.00065708
  67.68  -9.26  15.32 1003.88  0.00044106
...
```

```
track_id, year, month, day, hour, i, j, lon, la
0, 1950, 1, 1, 6, 572, 320, 201.269531, -14.882
0, 1950, 1, 1, 12, 569, 318, 200.214844, -15.35
0, 1950, 1, 1, 18, 567, 317, 199.511719, -15.58
0, 1950, 1, 2, 0, 565, 320, 198.808594, -14.882
...
```

```
netcdf ff_trs.test_out {
dimensions:
    tracks = 703 ;
    record = UNLIMITED ; // (78 currently)
variables:
    int TRACK_ID(tracks) ;
        TRACK_ID:add_fld_num = 0 ;
        TRACK_ID:tot_add_fld_num = 0 ;
        TRACK_ID:loc_flags = "" ;
    int FIRST_PT(tracks) ;
```

- Different format for every algorithm

FAIR outputs: the problem

```
start      2  1950      1      4      0
    54.67  -22.15    16.65  999.23    0.00027446
    55.72  -24.49    17.38  997.83    0.00031078
start     13  1950      1      6      0
    67.32   -9.26    17.90 1002.11    0.00065708
    67.68   -9.26    15.32 1003.88    0.00044106
...
```

```
track_id, year, month, day, hour, i, j, lon, la
0, 1950, 1, 1, 6, 572, 320, 201.269531, -14.882
0, 1950, 1, 1, 12, 569, 318, 200.214844, -15.35
0, 1950, 1, 1, 18, 567, 317, 199.511719, -15.58
0, 1950, 1, 2, 0, 565, 320, 198.808594, -14.882
...
```

```
netcdf ff_trs.test_out {
dimensions:
    tracks = 703 ;
    record = UNLIMITED ; // (78 currently)
variables:
    int TRACK_ID(tracks) ;
        TRACK_ID:add_fld_num = 0 ;
        TRACK_ID:tot_add_fld_num = 0 ;
        TRACK_ID:loc_flags = "" ;
    int FIRST_PT(tracks) ;
```

- Different format for every algorithm
- Unclear what each value represents

FAIR outputs: the problem

```
start      2  1950      1      4      0
    54.67  -22.15    16.65  999.23    0.00027446
    55.72  -24.49    17.38  997.83    0.00031078
start     13  1950      1      6      0
    67.32   -9.26    17.90 1002.11    0.00065708
    67.68   -9.26    15.32 1003.88    0.00044106
...
```

```
track_id, year, month, day, hour, i, j, lon, la
0, 1950, 1, 1, 6, 572, 320, 201.269531, -14.882
0, 1950, 1, 1, 12, 569, 318, 200.214844, -15.35
0, 1950, 1, 1, 18, 567, 317, 199.511719, -15.58
0, 1950, 1, 2, 0, 565, 320, 198.808594, -14.882
...
```

```
netcdf ff_trs.test_out {
dimensions:
    tracks = 703 ;
    record = UNLIMITED ; // (78 currently)
variables:
    int TRACK_ID(tracks) ;
        TRACK_ID:add_fld_num = 0 ;
        TRACK_ID:tot_add_fld_num = 0 ;
        TRACK_ID:loc_flags = "" ;
    int FIRST_PT(tracks) ;
```

- Different format for every algorithm
- Unclear what each value represents
- Limited metadata

FAIR outputs: the problem

```
start      2  1950      1      4      0
    54.67  -22.15    16.65  999.23    0.00027446
    55.72  -24.49    17.38  997.83    0.00031078
start     13  1950      1      6      0
    67.32   -9.26    17.90 1002.11    0.00065708
    67.68   -9.26    15.32 1003.88    0.00044106
...
```

```
track_id, year, month, day, hour, i, j, lon, la
0, 1950, 1, 1, 6, 572, 320, 201.269531, -14.882
0, 1950, 1, 1, 12, 569, 318, 200.214844, -15.35
0, 1950, 1, 1, 18, 567, 317, 199.511719, -15.58
0, 1950, 1, 2, 0, 565, 320, 198.808594, -14.882
...
```

```
netcdf ff_trs.test_out {
dimensions:
    tracks = 703 ;
    record = UNLIMITED ; // (78 currently)
variables:
    int TRACK_ID(tracks) ;
        TRACK_ID:add_fld_num = 0 ;
        TRACK_ID:tot_add_fld_num = 0 ;
        TRACK_ID:loc_flags = "" ;
    int FIRST_PT(tracks) ;
```

- Different format for every algorithm
- Unclear what each value represents
- Limited metadata
- Cannot reproduce

FAIR outputs: standardised format

All TCTrack outputs are converted into CF-Compliant NetCDF.

Again build on existing software by heavily relying on cf-python (Thanks to the NCAS RSE team!)

```
netcdf my_tracks {  
  dimensions:  
    trajectory = 36 ;  
    observation = 60 ;  
  variables:  
    int64 trajectory(trajectory) ;  
        trajectory:standard_name = "trajectory" ;  
        trajectory:cf_role = "trajectory_id" ;  
        trajectory:long_name = "trajectory index" ;  
    int64 observation(observation) ;  
        observation:standard_name = "observation" ;  
        observation:long_name = "observation index" ;  
    double time(trajectory, observation) ;  
        time:standard_name = "time" ;  
        time:long_name = "time" ;  
        time:units = "days since 1970-01-01" ;  
        time:calendar = "gregorian" ;  
}
```

FAIR outputs: metadata

Included metadata:

FAIR outputs: metadata

Included metadata:

- Variable metadata (copied from input files)

FAIR outputs: metadata

Included metadata:

- Variable metadata (copied from input files)
- TCTrack metadata

FAIR outputs: metadata

Included metadata:

- Variable metadata (copied from input files)
- TCTrack metadata
 - Version

FAIR outputs: metadata

Included metadata:

- Variable metadata (copied from input files)
- TCTrack metadata
 - Version
 - Tracking algorithm

FAIR outputs: metadata

Included metadata:

- Variable metadata (copied from input files)
- TCTrack metadata
 - Version
 - Tracking algorithm
 - Tracking parameters

FAIR outputs: reproducibility

The metadata is machine-readable, further simplifying reproduction:

```
from tctrack.utils import load_tracker_metadata
tracker, params = load_tracker_metadata("tracks_output.nc")
tracker.run_tracker("tracks_output_2.nc")
```

Further work - Visualisation dashboard

Standardised outputs enable downstream tooling.

For example a visualisation dashboard (in progress).

Further work - Visualisation dashboard

Standardised outputs enable downstream tooling.

For example a visualisation dashboard (in progress).

First we create a database from the outputs:

```
1 select * from observation_view where wind_speed > 30 order by wind_speed desc;
```

	file_id	trajectory_id	filename	date	latitude	latitude:1	air_pressure_at_sea_level	surface_altitude	wind_speed
1	1	674	cf_tracks_1950-2014.nc	1957-11-15 03:00:00	42.500008	42.500008	93746.69	0.0	34.0413
2	1	2573	cf_tracks_1950-2014.nc	1981-01-21 15:00:00	57.500008	57.500008	95337.44	0.0	33.16382
3	1	674	cf_tracks_1950-2014.nc	1957-11-14 21:00:00	41.388893	41.388893	93925.94	0.0	32.89518
4	1	2668	cf_tracks_1950-2014.nc	1982-01-23 03:00:00	43.611122	43.611122	94647.56	0.0	32.86119
5	2	9591	6hr_track_mm.nc	2003-08-02 12:00:00	23.611115	23.611115	96367.75	0.0	32.49823
6	1	1750	cf_tracks_1950-2014.nc	1971-03-19 15:00:00	-67.5	-67.5	97170.25	0.0	32.28371
7	2	10015	6hr_track_mm.nc	2008-10-10 18:00:00	33.055565	33.055565	97070.81	0.0	32.19182
8	1	1750	cf_tracks_1950-2014.nc	1971-03-19 09:00:00	-68.055557	-68.055557	97375.81	0.0	31.96919
9	2	8158	6hr_track_mm.nc	1985-09-06 06:00:00	24.722229	24.722229	96265.81	0.0	31.84956
10	2	8907	6hr_track_mm.nc	1994-10-28 00:00:00	19.166672	19.166672	97424.0	0.0	31.67181
11	2	7118	6hr_track_mm.nc	1972-10-14 06:00:00	16.94445	16.94445	97402.62	0.0	31.60194
12	2	9283	6hr_track_mm.nc	1999-09-11 18:00:00	20.277786	20.277786	96919.12	0.0	31.4941
13	2	9591	6hr_track_mm.nc	2003-08-02 18:00:00	24.722229	24.722229	96020.38	0.0	31.29167
14	2	7488	6hr_track_mm.nc	1977-07-24 12:00:00	23.611115	23.611115	96601.81	0.0	31.24293
15	1	3219	cf_tracks_1950-2014.nc	1988-11-05 21:00:00	38.055565	38.055565	96688.31	0.0	31.14808
16	2	7488	6hr_track_mm.nc	1977-07-25 06:00:00	26.94445	26.94445	96278.5	0.0	31.06671
17	2	5524	6hr_track_mm.nc	1952-09-30 00:00:00	31.388893	31.388893	97317.12	0.0	31.00851

Further work - Visualisation dashboard

Standardised outputs enable downstream tooling.

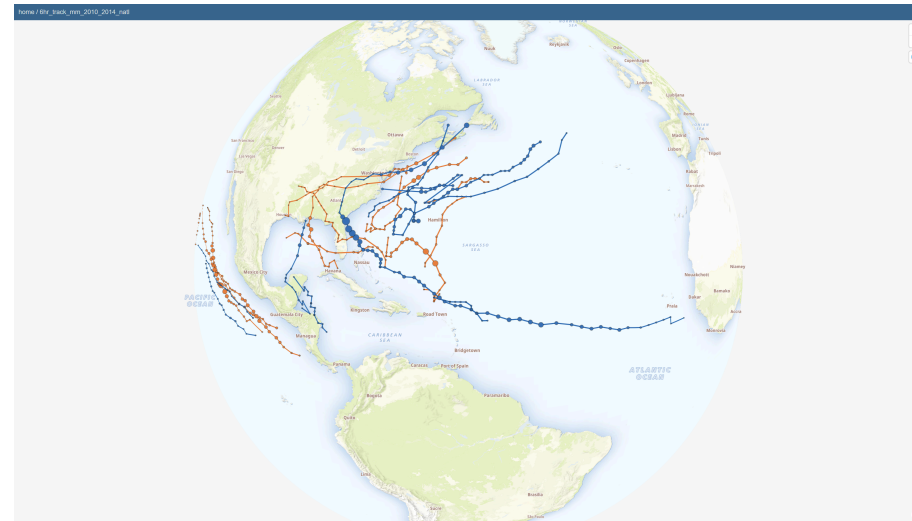
For example a visualisation dashboard (in progress).

First we create a database from the outputs:

```
1 select * from observation_view where wind_speed > 30 order by wind_speed desc;
```

	file_id	trajectory_id	filename	date	latitude	latitude:1	air_pressure_at_sea_level	surface_altitude	wind_speed
1	1	674	cf_tracks_1950-2014.nc	1957-11-15 03:00:00	42.500008	42.500008	93746.69	0.0	34.0413
2	1	2573	cf_tracks_1950-2014.nc	1981-01-21 15:00:00	57.500008	57.500008	95337.44	0.0	33.16382
3	1	674	cf_tracks_1950-2014.nc	1957-11-14 21:00:00	41.388893	41.388893	93925.94	0.0	32.89518
4	1	2668	cf_tracks_1950-2014.nc	1982-01-23 03:00:00	43.611122	43.611122	94647.56	0.0	32.86119
5	2	9591	6hr_track_mm.nc	2003-08-02 12:00:00	23.611115	23.611115	96367.75	0.0	32.49823
6	1	1750	cf_tracks_1950-2014.nc	1971-03-19 15:00:00	-67.5	-67.5	97170.25	0.0	32.28371
7	2	10015	6hr_track_mm.nc	2008-10-10 18:00:00	33.055565	33.055565	97070.81	0.0	32.19182
8	1	1750	cf_tracks_1950-2014.nc	1971-03-19 09:00:00	-68.055557	-68.055557	97375.81	0.0	31.96919
9	2	8158	6hr_track_mm.nc	1985-09-06 06:00:00	24.722229	24.722229	96265.81	0.0	31.84956
10	2	8907	6hr_track_mm.nc	1994-10-28 00:00:00	19.166672	19.166672	97424.0	0.0	31.67181
11	2	7118	6hr_track_mm.nc	1972-10-14 06:00:00	16.94445	16.94445	97402.62	0.0	31.60194
12	2	9283	6hr_track_mm.nc	1999-09-11 18:00:00	20.277786	20.277786	96919.12	0.0	31.4941
13	2	9591	6hr_track_mm.nc	2003-08-02 18:00:00	24.722229	24.722229	96020.38	0.0	31.29167
14	2	7488	6hr_track_mm.nc	1977-07-24 12:00:00	23.611115	23.611115	96601.81	0.0	31.24293
15	1	3219	cf_tracks_1950-2014.nc	1988-11-05 21:00:00	38.055565	38.055565	96688.31	0.0	31.14808
16	2	7488	6hr_track_mm.nc	1977-07-25 06:00:00	26.94445	26.94445	96278.5	0.0	31.06671
17	2	5524	6hr_track_mm.nc	1952-09-30 00:00:00	31.388893	31.388893	97317.12	0.0	31.00851

Then produce a dashboard with stats and visualisations:



Conclusions

Conclusions

1. **RSE support** helped to create a more useful software for the community.

Conclusions

1. **RSE support** helped to create a more useful software for the community.
2. **Domain knowledge** helped us improve ease of use for researchers' workflows.

Conclusions

1. **RSE support** helped to create a more useful software for the community.
2. **Domain knowledge** helped us improve ease of use for researchers' workflows.
3. **FAIR outputs** improve the (re)usability of research outputs.

Thank you!

✉ sa2329@cam.ac.uk

🐙 [Cambridge-ICCS/TCTrack](#)

📖 tctrack.readthedocs.io

